

I/O Management and Disk Scheduling



I/O Devices

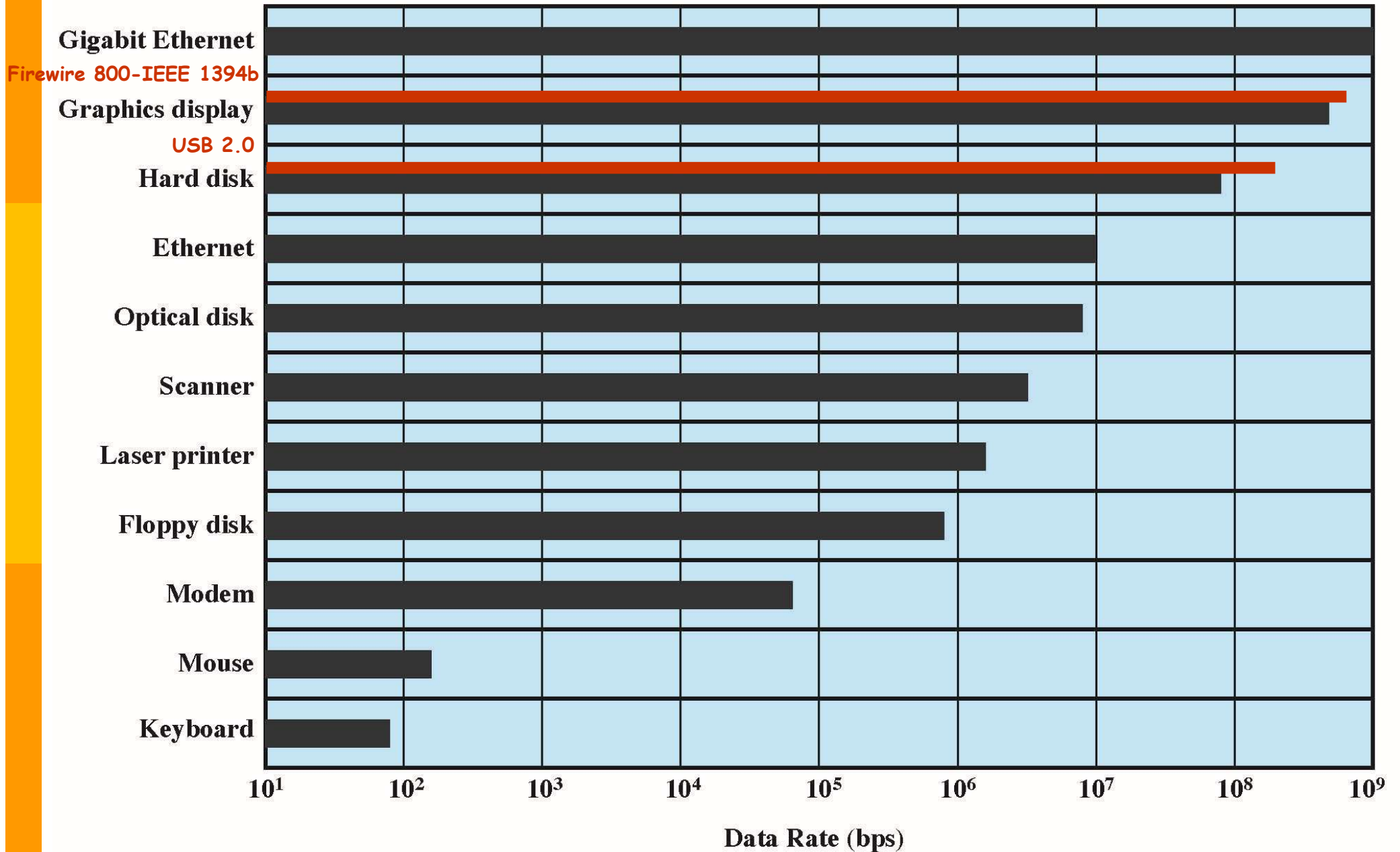
Categories:

- ❑ **For Human interaction:** Printers, terminals, keyboard, mouse
- ❑ **Machine readable:** Disks, USB, Sensors, Controllers, Actuators ...
- ❑ **Communication:** Modems, Network cards, etc.

Differences:

- ❑ **Data rate:** several orders of magnitude difference between transfer rates
- ❑ **Applications:** the use of the device influence the policies and software of OS e.g. Disk - store files, databases, virtual pages for MMU
- ❑ **Complexity of control:** e.g. Printer – simple control interface; disk – more complex
- ❑ **Unit of transfer:** e.g. stream of bytes for a terminal or in blocks for a disk
- ❑ **Data representation:** Different encoding schemes (e.g. parity conventions)
- ❑ **Error conditions:** how reported, consequences, range of response

Typical I/O Device Data Rates



Techniques for performing I/O

□ Programmed I/O

- Processor issues an I/O command, on behalf of a process, to an I/O module
- Process then **busy waits** for the operation to be completed before proceeding

□ Interrupt-driven I/O

- Processor issues an I/O command on behalf of a process
 - ▶ If I/O request is non-blocking: Processor continues executing instructions from the process that issued I/O
 - ▶ If I/O request is blocking: OS places process in blocked state, schedule another process
- I/O module sends an interrupt when done

□ Direct Memory Access (DMA)

- DMA controls exchange of data between main memory and I/O device
 - ▶ CPU sends request for transfer of a block of data to the DMA module
 - ▶ Processor is interrupted only after entire block has been transferred

Evolution of the I/O Function

1. **Processor** directly controls a peripheral device
2. **Controller** or **I/O module** is added
 - Processor uses programmed I/O without interrupts
 - Processor does not need to handle details of external devices
3. **Controller or I/O module with interrupts**
 - Efficiency improves as processor does not spend time waiting for an I/O operation to be performed
4. **Direct Memory Access:** I/O module is given direct control of memory via DMA
 - Blocks of data are moved into memory without involving the processor
 - Processor involved at beginning and end only
5. I/O module is a **separate processor**
 - CPU directs I/O processor to execute an I/O program in main memory.
6. **I/O processor**
 - I/O module has its own local memory
 - Commonly used to control communications with interactive terminals

Direct Memory Address

- ❑ Processor delegates I/O operation to the DMA module by sending
 - ❑ R/W requests, using R/W control lines between CPU and DMA
 - ❑ Address of IO device, thru data lines
 - ❑ Starting location in memory to read or write from, via data lines and stored by DMA in its Address Register
 - ❑ # words to be R/W, via data lines and stored in Data Count register
- ❑ CPU continues with other work
- ❑ DMA module transfers data directly to or from memory, over a bus, without going through CPU
- ❑ When complete DMA module sends an interrupt signal to the processor

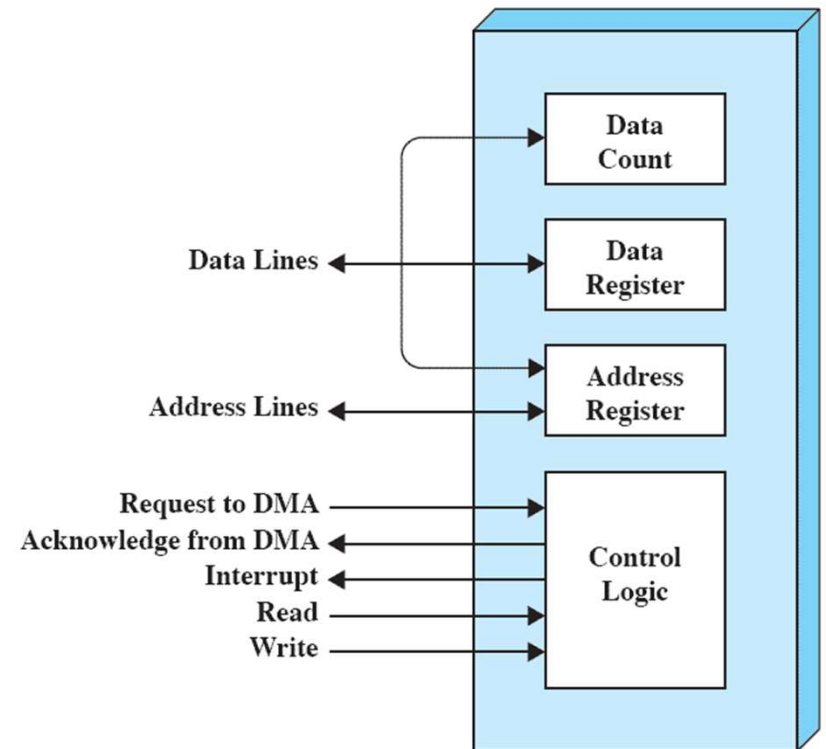
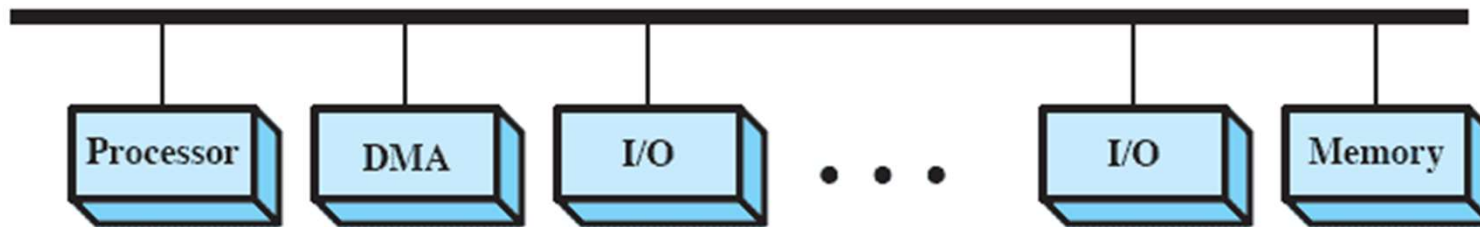


Figure 11.2 Typical DMA Block Diagram

DMA Configurations: Single Bus

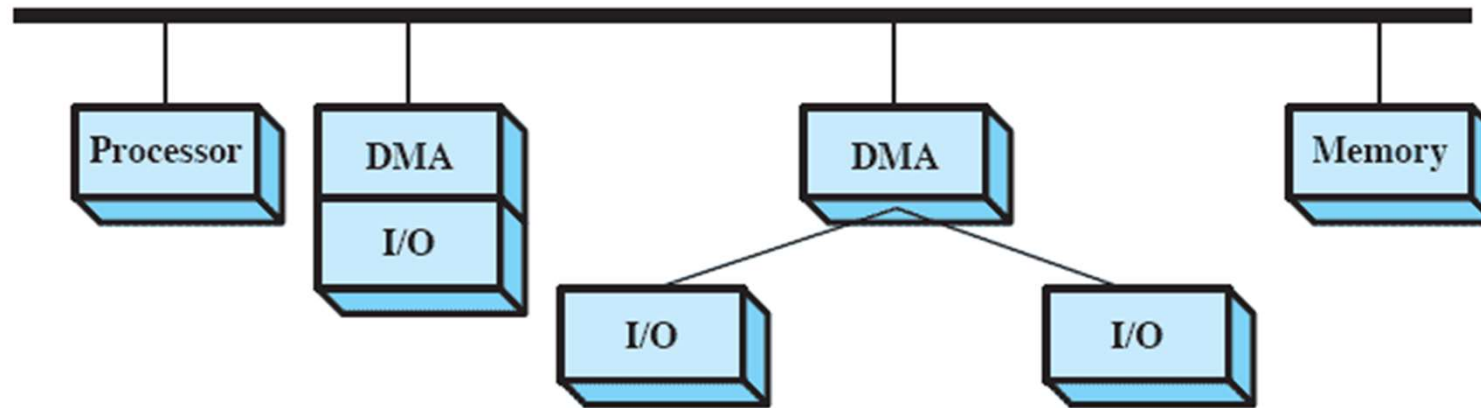
DMA can be configured in several ways



(a) Single-bus, detached DMA

- Single Bus: all modules share the same system bus
 - The DMA module, acting as a surrogate processor, uses programmed I/O to exchange data between memory and an I/O module through the DMA module
 - **Inexpensive** but **Inefficient**: with processor-controlled programmed I/O, each transfer of a word consumes two bus cycles (transfer request followed by transfer)

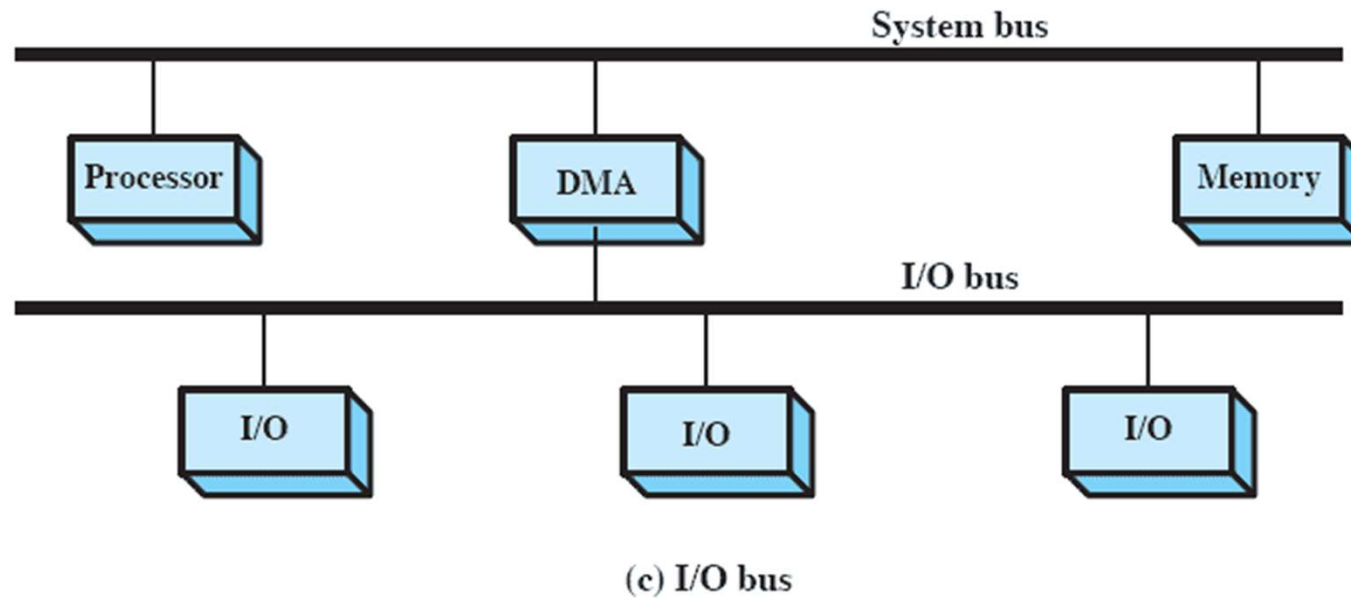
DMA Configurations: Integrated DMA & I/O



(b) Single-bus, Integrated DMA-I/O

- Integrating DMA and I/O functions
 - Direct Path between DMA and I/O modules, not including system bus
- This substantially cuts the required bus cycles

DMA Configurations: I/O Bus



- connecting I/O modules to DMA module using an I/O bus
- Reduces the number of I/O interfaces in the DMA module
- exchange of data between the DMA and I/O modules takes place off the system bus
- system bus is used by DMA module only to exchange data with memory and control signals with the processor

Operating System Design Issues

□ Efficiency

- Most I/O devices are extremely slow compared to main memory
- I/O cannot keep up with processor speed
- Use of *multiprogramming* allows for some processes to be waiting on I/O while another process executes
- *Swapping* is used to bring in additional ready processes
 - ▶ But: It is an I/O operation itself!

Major efforts in I/O design is to improve efficiency, especially Disk I/O

□ Generality

- Desirable to handle all I/O devices in a uniform manner
- Difficult to completely generalize, but can use a hierarchical modular design of I/O functions
 - ▶ Hide the details of device I/O in lower-level routines. Processes and upper levels see devices in general terms such as *read*, *write*, *open*, *close*, *lock*, *unlock*

I/O Buffering

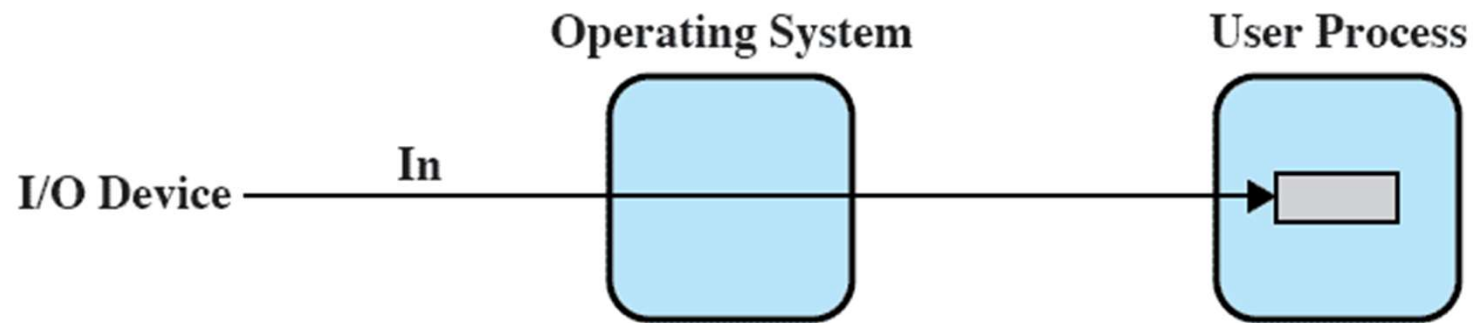
- Perform **input transfers** in **advance** of requests being made and to perform **output transfers** some time **after** the request is made.
- **Reasons** for buffering
 - Processes must wait for I/O to complete before proceeding
 - ▶ Overlap I/O with processing and increase **efficiency**
 - If *buffering* is not used, there is an **interference with swapping** decisions by a OS, and even **possibility of deadlock**
 - ▶ Consider a process issued I/O request and waiting for I/O to complete
 - ▶ Pages involved in the I/O transfer must remain in memory – locked
 - ▶ Portion of process can be paged out but process can not be swapped
 - ▶ If swapped out, a deadlock can occur
 - Process is blocked waiting for I/O event to complete
 - I/O is blocked waiting for process to be swapped in

I/O Buffering

- Distinguish between two types of I/O devices
- **Block-oriented Buffering**
 - Information is stored in fixed sized blocks
 - Transfers are made a block at a time
 - ▶ Can reference data by block number
 - Used for disks and USB keys
- **Stream-Oriented Buffering**
 - Transfer information as a stream of bytes
 - Used for terminals, printers, communication ports, mouse and other pointing devices, and most other devices that are not secondary storage

No Buffer

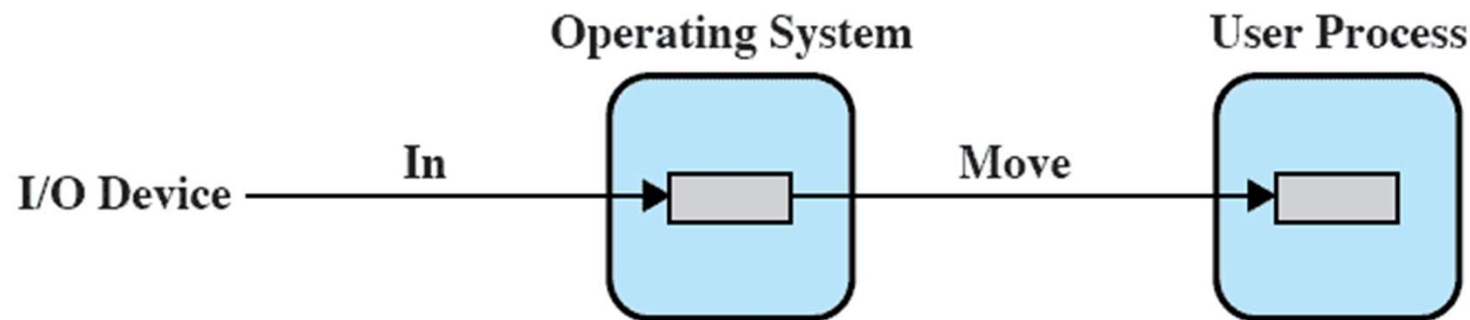
- Without a buffer, the OS directly access the device as and when it needs



(a) No buffering

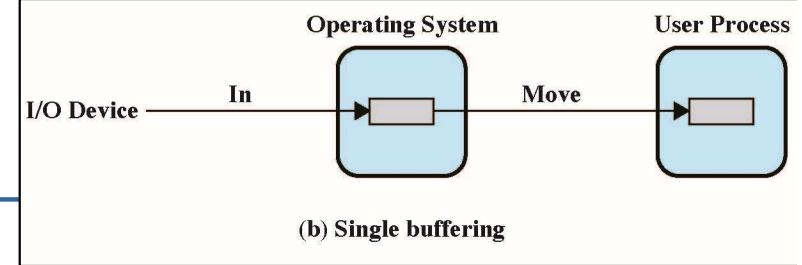
Single Buffer

- ❑ When a user issues IO request, OS assigns a **system buffer** in main memory for the I/O request
- ❑ Input transfers are made to system buffer
- ❑ Block is moved to **user space** when transfer is complete
- ❑ The next block is moved into the buffer
 - ❑ Called: **Read ahead** or Anticipated Input
 - ❑ Often a reasonable assumption as data is usually accessed sequentially



(b) Single buffering

Single Buffer



□ Performance

M = time to move the data into user space

T = Disk I/O Time to input one block

C = Computation Time that intervenes between input requests

No Buffer →→ Execution Time = $T + C$

Single Buffer →→ Execution Time = **Maximum** [C, T] + M

(overlap I/O with computation)

□ Advantage

□ **Speedup:** User process processes one data block while next block is read in

□ Swapping can occur since input is taking place in system memory, not user memory

□ Overhead:

□ Complicate the logic in OS: OS must keep track of assignment of system buffers to user processes

Double Buffer

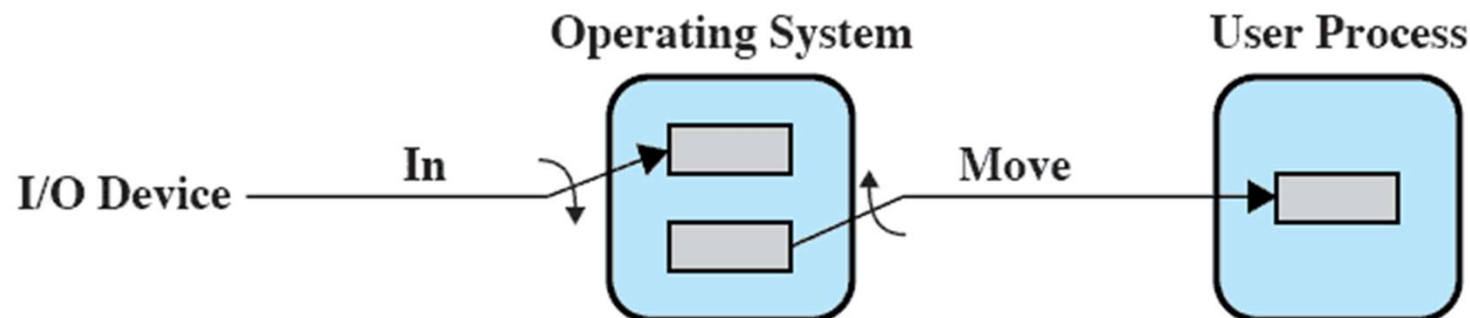
- Use two system buffers instead of one
- A process can transfer data to or from one buffer while the operating system empties or fills the other buffer

T = Disk I/O Time **C** = Computation Time between requests

M = time to move the data into user space

If $(C + M) < T \rightarrow$ **I/O device** is working at **full speed**

If $(C + M) > T \rightarrow$ **Process will not have to wait for IO,**
add more I/O buffers to increase efficiency!



(c) Double buffering

Circular Buffer

- ❑ Double buffering may be inadequate if the process performs rapid bursts of I/O
 - More than two buffers are used for IO to keep up with the process
- ❑ Each individual buffer is one unit in a circular buffer
 - ❑ Bounded-buffer producer/consumer problem



(d) Circular buffering

I/O Single Buffer – Stream-oriented I/O

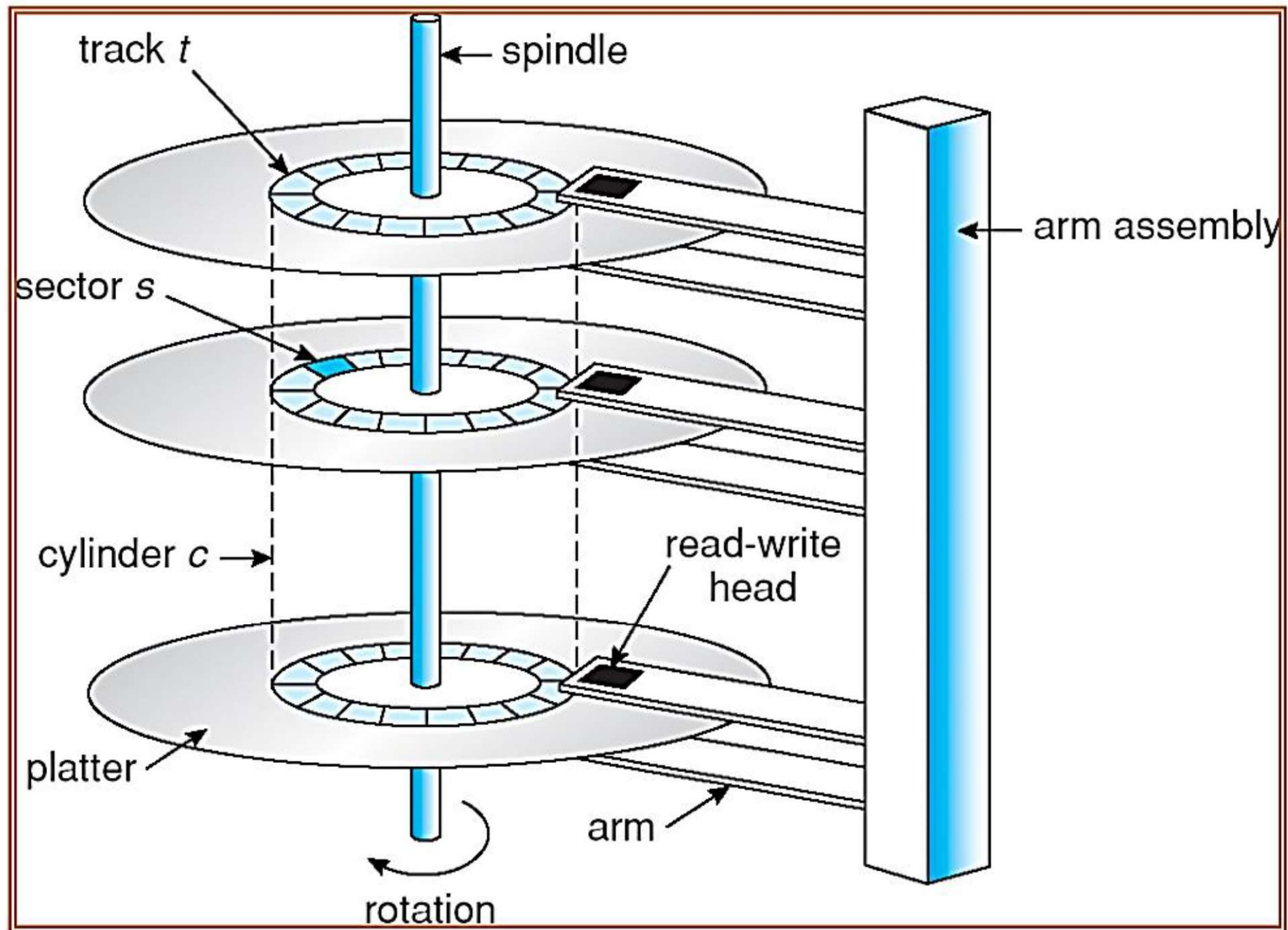
□ line-at-a time I/O

- used on scroll-mode terminals: user input is one line at a time, with a carriage return signaling end of a line. A line printer is another example
- the buffer can be used to hold a single line
- user process is suspended during input, awaiting the arrival of the entire line
- For output, the user process can place a line in the buffer and continue processing. It need not be suspended unless it has a second line of output to send before the buffer is emptied from the first output operation

□ byte-at-a-time I/O

- is used on forms-mode terminals, when each keystroke is significant, and for many other peripherals, such as sensors and controllers.
- the interaction between OS and the user process follows the **producer/consumer model**

Disk Components

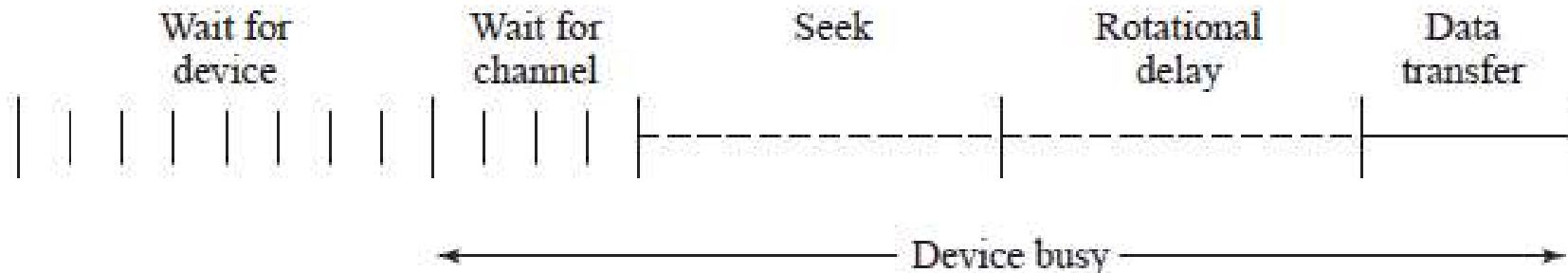


Disk Performance Parameters

- To read/write: **disk head** must be positioned at the right **track** and **sector**
- **Seek time**: position the head at the desired track
- **Rotational delay (latency)**: time taken for start of the sector to reach the head

$$\text{Access time} = \text{seek time} + \text{rotational delay}$$

- **Data transfer** time taken to transfer data; occurs as sector moves under the head



Disk Parameters in 2015

- **Seek times**: 1-15ms, depending on distance (avg. 5-6ms - improving at 7-10% per year)
- **Rotation speeds**: 10,000 RPMs (avg. 3ms - improving at 7-10% per year)
- **Data rates**: 100-300 MB/s, depending on zone
 - Avg. sector transfer time of 100-200μs (improving at 40% per year)

Disk Performance

□ Transfer time $T = \frac{b}{rN}$

where

T = transfer time

b = number of bytes to be transferred

N = number of bytes on a track

r = rotation speed, in revolutions per second

□ Total average access time

$$T_a = T_s + \frac{1}{2r} + \frac{b}{rN}$$

- the **order** in which sectors are read from the disk has a tremendous effect on I/O performance.

Timing Comparison

- Consider a disk with
 - avg seek time of 4ms, rotation speed of 7500 rpm ($7500/60 = 125$ rps),
 - 512-byte sectors with 500 sectors per track
- Suppose that we wish to read a file consisting of 2500 sectors for a total of 1.28 Mbytes. What is the total time for the transfer?
- **sequential organization**: file occupies all of the sectors on 5 adjacent tracks
 - time to read first track =
 - ▶ seek time = 4 ms
 - ▶ rotational delay = 4ms [$1/2r = 1/(2 \times 125) = 0.004$]
 - ▶ read 500 sectors = 8ms [$b/rN = (500 \times 512)/(125 \times 500 \times 512) = 16$ ms]
 - Due to contiguous storage, next tracks do not need seek time, only rotational and transfer = $4 + 8 = 12$ ms
 - Total time = $16 + (4 * 12) = 64$ ms = **0.064 seconds**
- **Random access**: accesses to the sectors are distributed randomly over disk
 - For each sector: seek (4ms) + Rotational delay (4ms) + Read 1 sector (0.016 ms) = 8.016 ms
 - Total time = $2500 * 8.016 = 20,040$ ms = **20.04 seconds**

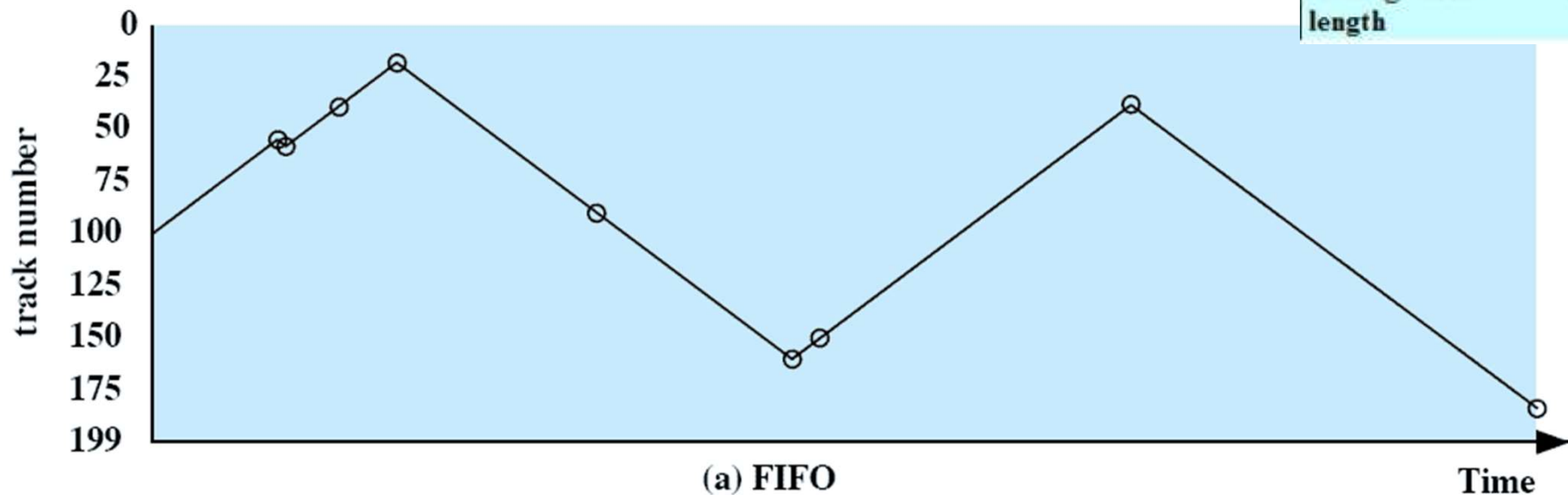
Disk Scheduling Policies

- For a single disk there will be a number of I/O requests
- If requests are serviced randomly → worst possible performance
- **Access time** – especially **seek** time - is the reason for differences in performance
- To improve performance, we need to reduce the average time spent on seeks
- To compare various schemes, consider a disk head is initially located at track 100.
 - assume a disk with 200 tracks and that the disk request queue has random requests in it
- The requested tracks, in the order received by the disk scheduler, are
 - 55, 58, 39, 18, 90, 160, 150, 38, 184

Disk Scheduling Policies: FIFO

- Process request sequentially
- **Fair** to all processes
- Reasonable performance if
 - only few processes and
 - requests are to clustered file sectors
- Approaches random scheduling in performance if there are many processes
- Ex: 55, 58, 39, 18, 90, 160, 150, 38, 184 start@ 100

(a) FIFO (starting at track 100)	
Next track accessed	Number of tracks traversed
55	45
58	3
39	19
18	21
90	72
160	70
150	10
38	112
184	146
Average seek length	55.3



Disk Scheduling Policies: Priority, LIFO

□ **Priority**

- Goal: not to optimize disk use but to meet other objectives
- Short batch jobs may have higher priority
 - ▶ Provide good interactive response time
 - ▶ Longer jobs may have to wait an excessively long time
- A poor policy for database systems

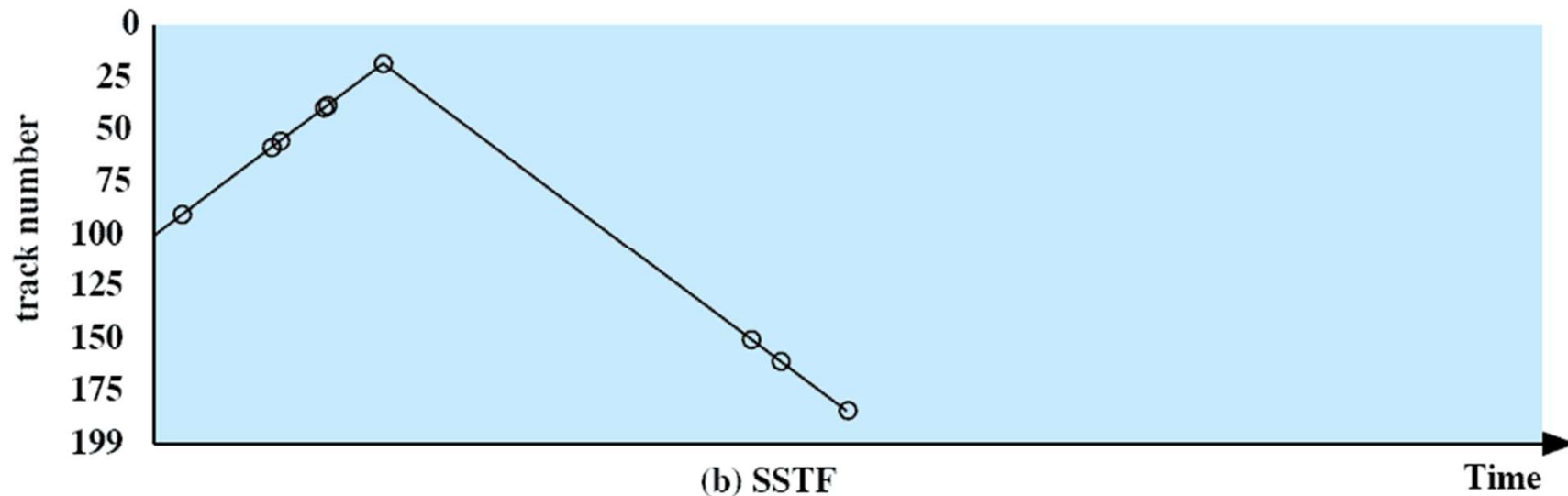
□ **Last-In First-Out**

- Exploits locality of reference
 - ▶ The device is given to the most recent user so there should be little arm movement
 - ▶ Improves throughput and reduces queues lengths
 - ▶ Good for transaction processing systems
- Possibility of starvation since a job may never regain the head of the line

Shortest Service Time First

- Select the disk I/O request that requires the **least movement of the disk arm** from its current position
- Always choose the minimum seek time
 - does not guarantee that the average seek time over a number of arm movements will be minimum
- Better performance than FIFO
- Ex: 55, 58, 39, 18, 90, 160, 150, 38, 184

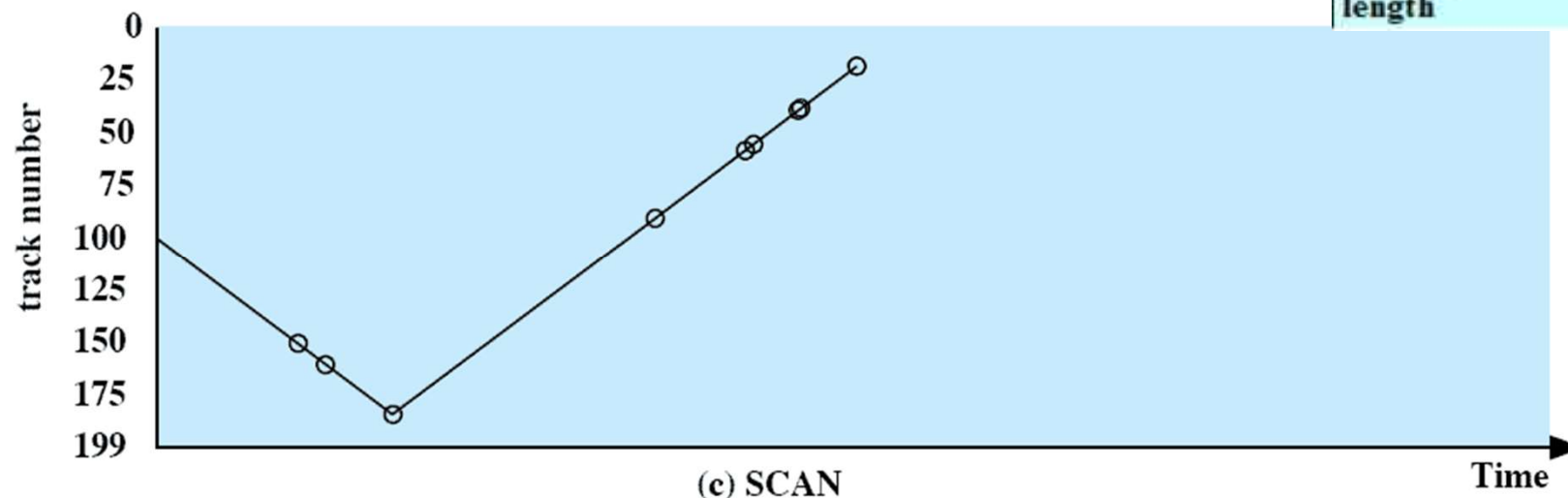
(b) SSTF (starting at track 100)	
Next track accessed	Number of tracks traversed
90	10
58	32
55	3
39	16
38	1
18	20
150	132
160	10
184	24
Average seek length	27.5



SCAN

- Arm moves in one direction only, satisfying all outstanding requests until it reaches the last track in that direction
- then the direction is reversed
- Also known as **Elevator** or **LOOK** policy
- $\approx$ SSTF, but avoids starvation of new requests
- Favors
 - Jobs whose requests are for tracks nearest to both innermost and outermost tracks
 - latest-arriving jobs
- Ex: 55, 58, 39, 18, 90, 160, 150, 38, 184

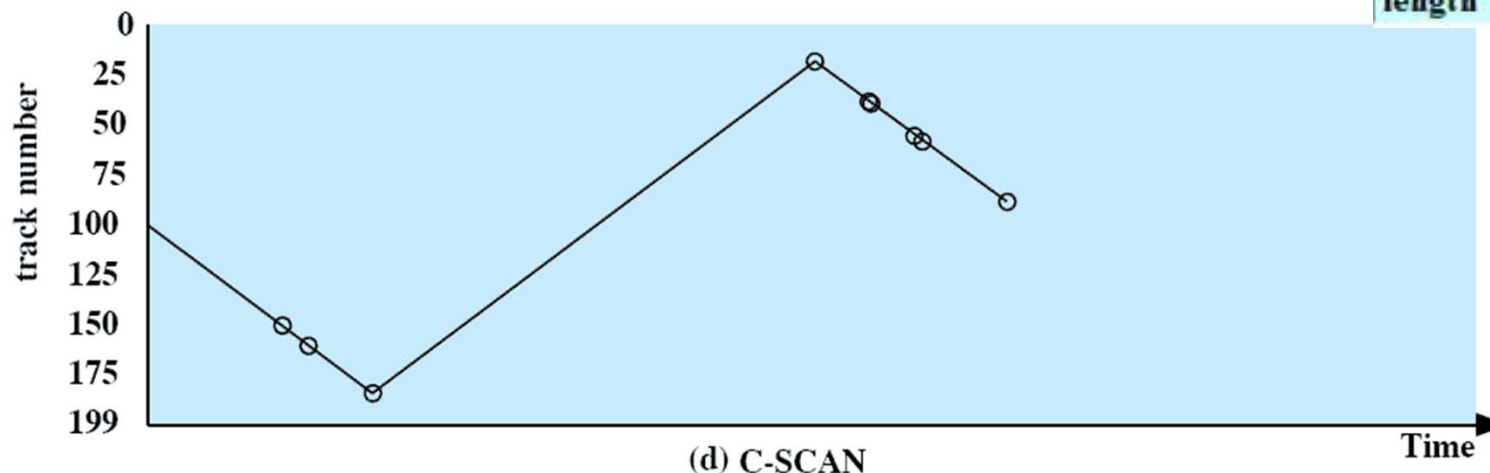
(c) SCAN (starting at track 100, in the direction of increasing track number)	
Next track accessed	Number of tracks traversed
150	50
160	10
184	24
90	94
58	32
55	3
39	16
38	1
18	20
Average seek length	27.8



C-SCAN (Circular SCAN)

- ❑ Restricts scanning to **one direction** only
- ❑ When the last track has been visited in one direction, the arm is returned to the opposite end of the disk and the scan begins again
- ❑ This reduces the maximum delay experienced by **new requests**.
- ❑ Reduces delays for sectors at **periphery**

(d) C-SCAN (starting at track 100, in the direction of increasing track number)	
Next track accessed	Number of tracks traversed
150	50
160	10
184	24
18	166
38	20
39	1
55	16
58	3
90	32
Average seek length	35.8



Comparison of Disk Scheduling Algorithms

(a) FIFO (starting at track 100)		(b) SSTF (starting at track 100)		(c) SCAN (starting at track 100, in the direction of increasing track number)		(d) C-SCAN (starting at track 100, in the direction of increasing track number)	
Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed	Next track accessed	Number of tracks traversed
55	45	90	10	150	50	150	50
58	3	58	32	160	10	160	10
39	19	55	3	184	24	184	24
18	21	39	16	90	94	18	166
90	72	38	1	58	32	38	20
160	70	18	20	55	3	39	1
150	10	150	132	39	16	55	16
38	112	160	10	38	1	58	3
184	146	184	24	18	20	90	32
Average seek length	55.3	Average seek length	27.5	Average seek length	27.8	Average seek length	35.8

N-step-SCAN and FSCAN

- **Arm stickiness** with SSTF, SCAN and C-SCAN
 - The arm may not move for a considerable period of time
 - if one or more processes have high access rates to one track, they can monopolize the device
- **N-step-SCAN**
 - Segments the disk request queue into subqueues of length N
 - Subqueues are processed one at a time, using SCAN
 - New requests added to some other queue when a queue is processed
- **FSCAN**: uses two subqueues
 - When a scan begins, all of the requests are in one of the queues, with the other empty.
 - All new requests are put into the other queue.
 - Service of new requests is deferred until all of the old requests have been processed.

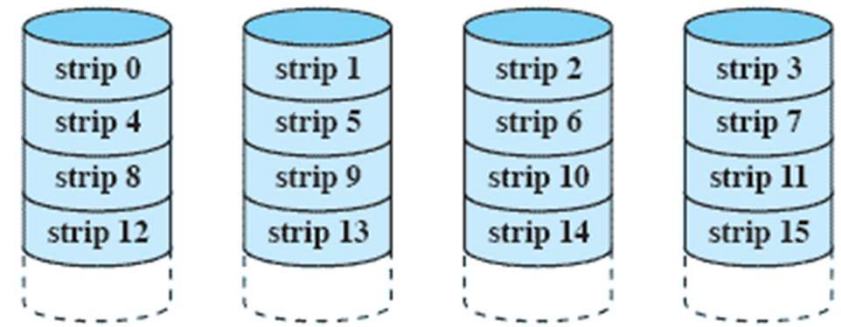
Disk Scheduling Algorithms

Name	Description	Remarks
Selection according to requestor		
RSS	Random scheduling	For analysis and simulation
FIFO	First in first out	Fairest of them all
PRI	Priority by process	Control outside of disk queue management
LIFO	Last in first out	Maximize locality and resource utilization
Selection according to requested item		
SSTF	Shortest service time first	High utilization, small queues
SCAN	Back and forth over disk	Better service distribution
C-SCAN	One way with fast return	Lower service variability
N-step-SCAN	SCAN of N records at a time	Service guarantee
FSCAN	N-step-SCAN with N = queue size at beginning of SCAN cycle	Load sensitive

RAID

- Rate of improvement of secondary storage performance << microprocessor
- Multiple Disks in Parallel
 - I/O requests can be executed in parallel, increasing I/O performance
 - Redundancy is added to improve reliability
- Redundant Array of Independent Disks (RAID)
 - Set of physical disk drives viewed by OS as a single logical drive
 - Data are distributed across the physical drives of an array
 - Redundant disk capacity is used to store parity information which provides recoverability from disk failure
- The RAID scheme consists of 7 levels, 0 through 6

RAID 0 - Striped

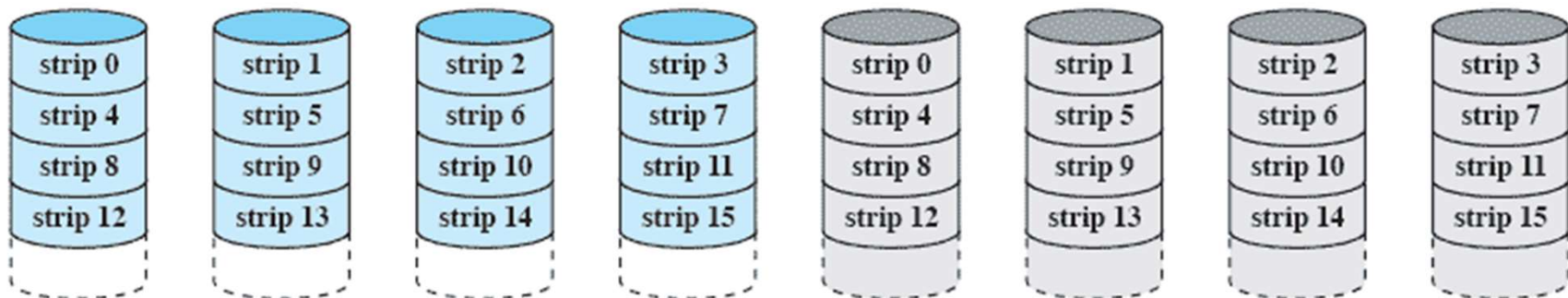


(a) RAID 0 (non-redundant)

- Files are striped across disks
- Strips are mapped Round Robin to consecutive physical disks
- Each stripe portion is read/written in parallel
- **Adv:**
 - if a single I/O request consists of multiple logically contiguous strips, then up to n strips for that request can be handled in parallel, improving performance
 - Also may balance load across disks
- Not a true RAID
 - No redundancy
 - Disk failure is catastrophic

RAID 1 - Mirrored

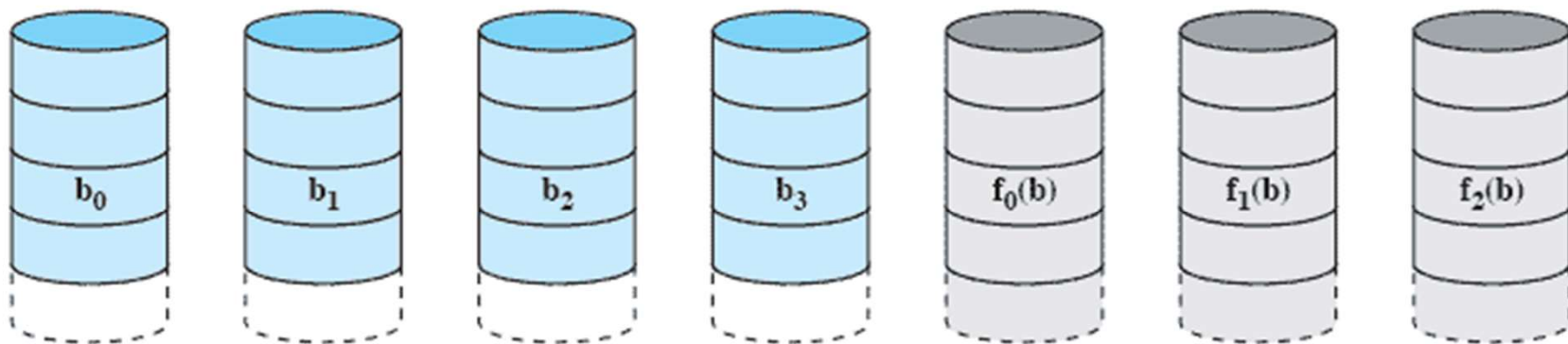
- ❑ *Redundancy* is achieved by *duplicating* all the data
- ❑ Each logical strip is mapped to *two* separate physical disks
- ❑ A *read request* can be serviced by either of the two disks
- ❑ A *write request* requires that both strips be updated; but can be done in parallel
- ❑ When a drive fails, data can be accessed from the mirror disk
- ❑ Performance: Higher I/O request than RAID 0, if bulk is “Read”
- ❑ *Disadvantage:* cost is doubled
- ❑ limited to highly critical data and system software



(b) RAID 1 (mirrored)

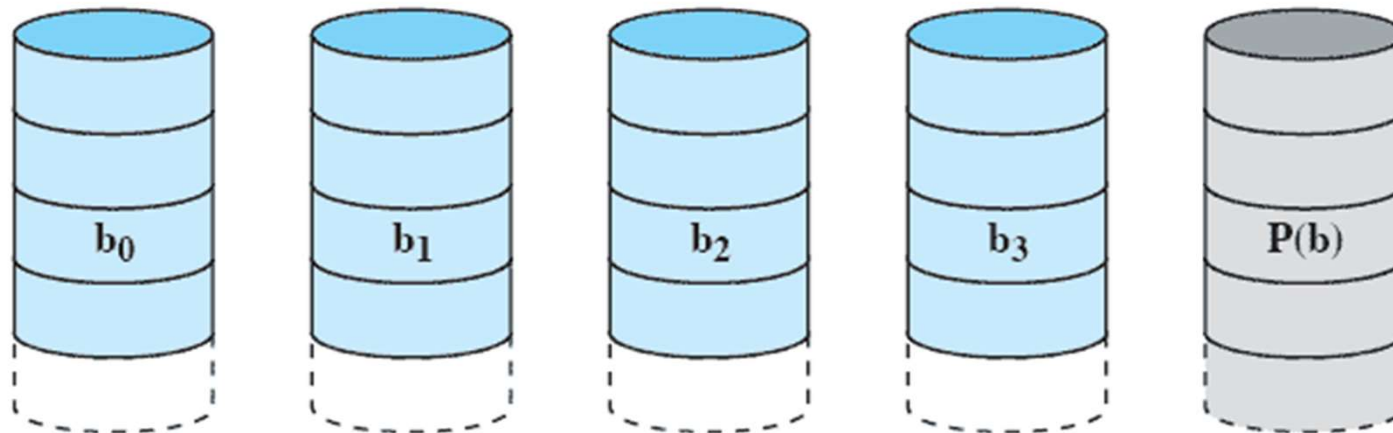
RAID 2 (Using Hamming code)

- Data stripping is used: The strips are small; a single byte or a word
- **All disks** participate in execution of each IO request: Synchronised disk rotation
- An **error-correcting code** is calculated across corresponding bits on each data disk, and bits of the code are stored in the corresponding bit positions on multiple **parity** disks.
- **Hamming code** is used to correct single bit errors and detect double-bit errors
- Costly; only useful when disk errors are very high



RAID 3 bit-interleaved parity

- ❑ Similar to RAID-2 but requires **Single redundant disk**
- ❑ Uses parallel access with data distributed in small strips.
- ❑ Uses **parity bits** instead of error correcting code
- ❑ A simple parity bit is computed for the set of individual bits in the same position on all of the disks
- ❑ **Performance**
 - ❑ ++ any IO request will involve parallel transfer from all disks → High transfer rate
 - ❑ -- only one request can be executed at a time → low IO request rate → transaction oriented environment suffers



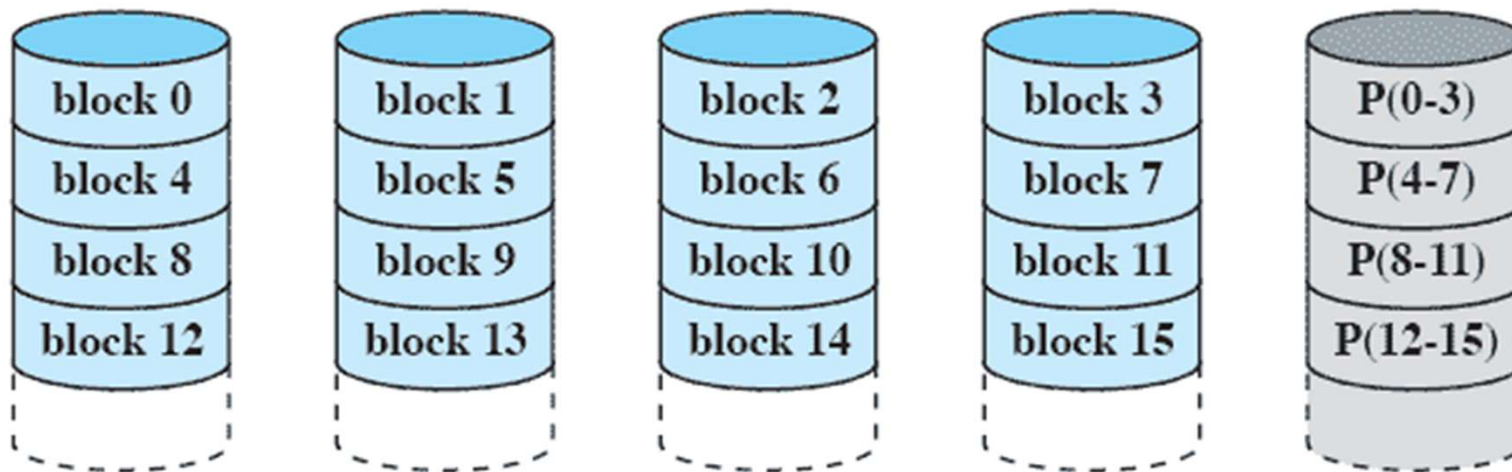
RAID 3 Data Reconstruction

Redundancy In the event of a drive failure, the parity drive is accessed and data is reconstructed from the remaining devices

- Assume 5 drives
 - ▶ $X0 - X3 = \text{data}$ $X4 = \text{parity}$
- Parity for i^{th} bit is:
 - ▶ $X4(i) = X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i)$
- Drive X1 fails
- Add $X4(i) \oplus X1(i)$ to both sides and we get:
 - ▶ $X1(i) = X3(i) \oplus X2(i) \oplus X4(i) \oplus X0(i)$
- If one disk fails, contents of each strip of data on X1 can be regenerated from the contents of the corresponding strips on the remaining disks

RAID 4 Block-level parity

- ❑ Disks operate independently
 - ❑ Separate IO requests can be satisfied in parallel
 - ❑ High IO requests rates
- ❑ A bit-by-bit parity strip is calculated across corresponding strips on each disk and the parity bits are stored in the corresponding strip on the parity disk.
- ❑ Write penalty: each write request must update parity bits

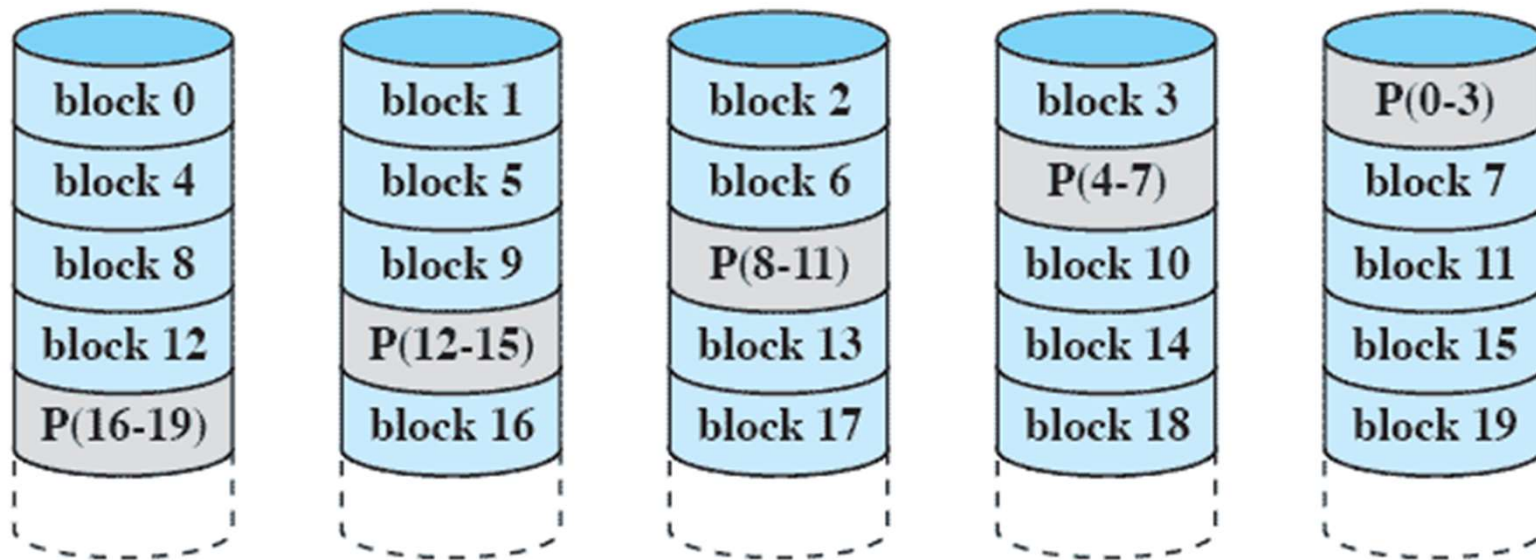


RAID 4 Parity Calculation

- Assume 5 drives $X0 - X3 = \text{data}$ $X4 = \text{parity}$
- Parity for i^{th} bit is $X4(i) = X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i)$
- A **write request** occurs that only affects a strip on $X1$
- What happens to the parity calculation?
- $X4'(i) = X3(i) \oplus X2(i) \oplus X1'(i) \oplus X0(i)$
 - Disks with altered bits have a prime
- $X4'(i) = X3(i) \oplus X2(i) \oplus X1'(i) \oplus X0(i) \oplus X1(i) \oplus X1(i)$
- $X4'(i) = X3(i) \oplus X2(i) \oplus X1(i) \oplus X0(i) \oplus X1(i) \oplus X1'(i)$
- $X4'(i) = X4(i) \oplus X1(i) \oplus X1'(i)$
- Read old user strip and old parity strips, then update
- Each strip write involves: 2 reads and 2 writes
- Bottleneck at the parity disk

RAID 5 Block-level Distributed parity

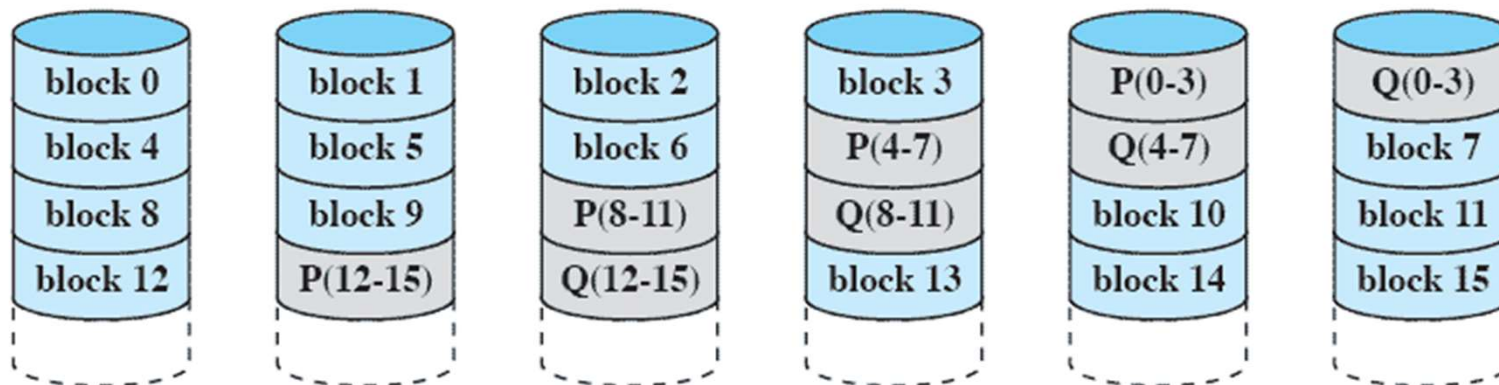
- Similar to RAID-4 but distributing the parity bits across all drives, usually using Round Robin
- Removes single parity disk bottleneck of RAID 4



(f) RAID 5 (block-level distributed parity)

RAID 6 - Dual Redundancy

- ❑ Two different parity calculations are carried out
 - ❑ stored in separate blocks on different disks
 - ❑ if user data require N disks, RAID 6 needs N+2 disks
- ❑ P and Q are two different data check algorithms
 - ❑ One is the exclusive-OR calculation used in RAID 4 and 5
 - ❑ The other is an independent data check algorithm.
- ❑ **Advantage:** high data availability: Can recover from two disks failing. Three disks would have to fail within the MTTR (mean time to repair) interval to cause data to be lost
- ❑ **Disadvantage:** incurs a substantial write penalty, because each write affects two parity blocks. Performance benchmarks shows 30% drop over RAID5



RAID levels comparison

Category	Level	Description	Disks required	Data availability	Large I/O data transfer capacity	Small I/O request rate
Striping	0	Nonredundant	N	Lower than single disk	Very high	Very high for both read and write
Mirroring	1	Mirrored	$2N$	Higher than RAID 2, 3, 4, or 5; lower than RAID 6	Higher than single disk for read; similar to single disk for write	Up to twice that of a single disk for read; similar to single disk for write
Parallel access	2	Redundant via Hamming code	$N + m$	Much higher than single disk; comparable to RAID 3, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
	3	Bit-interleaved parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 4, or 5	Highest of all listed alternatives	Approximately twice that of a single disk
Independent access	4	Block-interleaved parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 3, or 5	Similar to RAID 0 for read; significantly lower than single disk for write	Similar to RAID 0 for read; significantly lower than single disk for write
	5	Block-interleaved distributed parity	$N + 1$	Much higher than single disk; comparable to RAID 2, 3, or 4	Similar to RAID 0 for read; lower than single disk for write	Similar to RAID 0 for read; generally lower than single disk for write
	6	Block-interleaved dual distributed parity	$N + 2$	Highest of all listed alternatives	Similar to RAID 0 for read; lower than RAID 5 for write	Similar to RAID 0 for read; significantly lower than RAID 5 for write

N = number of data disks; m proportional to $\log N$

Disk Cache

- ❑ Disk cache is **buffer in main memory** for disk sectors
- ❑ Contains a copy of some of the sectors on the disk
- ❑ Handling a request
 - ❑ If the data is in cache, pass it to the user (copy or use shared memory)
 - ❑ If not in cache, read from the disk and add to the cache
- ❑ Because of phenomenon of **locality of reference**:
 - ❑ when a block of data is fetched, it is likely that there will be future references to that same block.
- ❑ A number of ways exist to populate the cache

Least Recently Used

- The block that has been in the cache the **longest with no reference** to it is **replaced**
- **Implementation**
 - The cache consists of a **stack** of blocks with Most recently referenced block on top
 - When a block is **referenced** or brought into the cache, it is placed on the **top** of the stack
 - The block on the **bottom** of the stack is **removed** when a new block is brought in
 - Blocks don't actually move around; **pointers** to the blocks are moved
- Most commonly used algorithm

Least Frequently Used

- The block that has experienced the **fewest references** is replaced
- **Implementation**
 - A **counter** is associated with each block
 - Counter is incremented each time block accessed
 - When replacement is required, the block with the smallest count is selected
- Some blocks may be referenced many times in a short period of time and the reference count is misleading

Unix System V R4 (SVR4) I/O

- ❑ Each I/O device associated with a special file (/dev/xxx)
 - ❑ Managed by the file system
 - ❑ Use ordinary read/write calls to access the device
 - ❑ Clean, uniform interface to users and processes
- ❑ Devices separated into
 - ❑ character devices : Terminals, printers,
 - ❑ Block devices: disks, tapes, etc..
- ❑ Two types of I/O
 - ❑ Buffered: passes thru two types of system buffers
 - ▶ Buffer Cache
 - ▶ Character Queue
 - ❑ Unbuffered
 - ▶ Involves DMA, with transfer taking place directly between IO module and process area

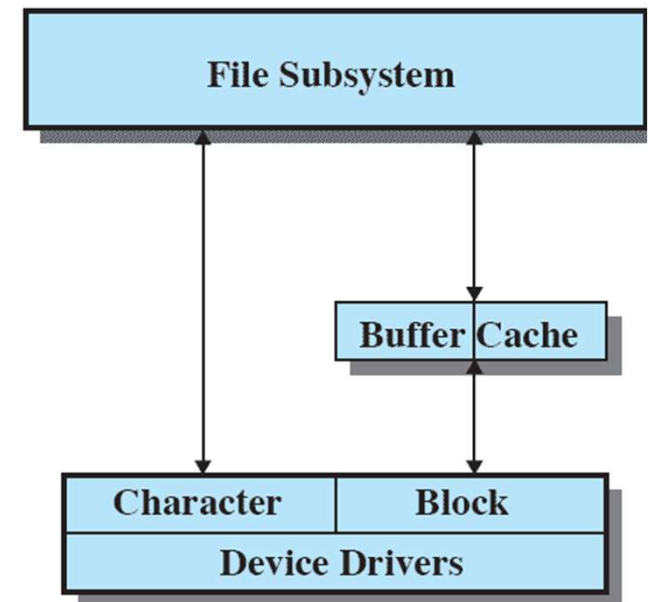


Figure 11.12 UNIX I/O Structure

Buffer Cache

- ❑ Serves as a disk cache
- ❑ data transfer from buffer cache to user process space is thru DMA
- ❑ To manage cache, maintains 3 lists
 - ❑ **Free List** – Available slots/buffers
 - ❑ **Device List** – buffers currently associated with a device
 - ❑ **Driver I/O Queue** - Buffers that are actually being used or waiting for I/O
- ❑ Use a hash table to quickly find the buffer corresponding to a particular device and block
- ❑ Uses LRU for blocks in the queue
 - ❑ Order kept by the free list order

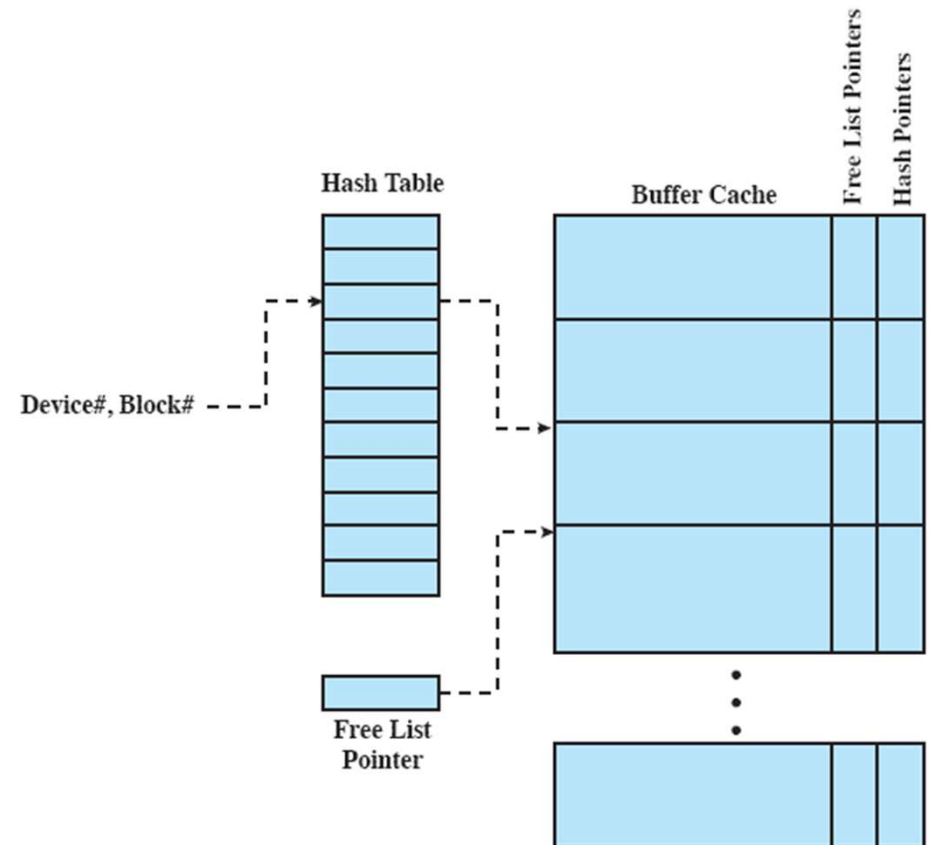


Figure 11.13 UNIX Buffer Cache Organization

Character Cache

- Used by character oriented devices
 - e.g. terminals and printers
- Either written by the I/O device and read by the process or vice versa
- Producer/consumer model used
 - Data from the device is written to a queue
 - Process reads data from the queue
 - Data “destroyed” as it is read
- contrast to the buffer cache, which may be read multiple times and hence follows the readers/writers model

Unbuffered I/O

- Unbuffered I/O is simply DMA between device and process space
 - Fastest method
 - Process is locked in main memory and cannot be swapped out
 - Device is tied to process for the duration of the transfer and unavailable for other processes

I/O for Device Types

Table 11.5 Device I/O in UNIX

	Unbuffered I/O	Buffer Cache	Character Queue
Disk drive	X	X	
Tape drive	X	X	
Terminals			X
Communication lines			X
Printers	X		X

Linux I/O

- Linux and Unix (e.g. SVR4) are very similar in I/O terms
 - The Linux kernel associates a special file with each I/O device driver
 - Block, character, and network devices are recognized
- Disk Scheduling
 - Variation of Elevator / LOOK algorithm (Linux 2.4)
 - ▶ Deadline scheduler (Linux 2.6)
 - ▶ Anticipatory IO Scheduler (Linux 2.6)

The Elevator Scheduler (SCAN/LOOK)

- Maintains a single queue for disk read and write requests
- Keeps list of requests sorted by block number
- Drive moves in a single direction to satisfy each request
- Problems with Elevator Scheduler
 - **Distant blocks can be delayed** as queue is dynamically updated
 - ▶ Consider requests: 20, 30, 700, 25
 - ▶ Reordered to: 20, 25, 30, 700
 - ▶ If a bunch of low number requests come in request for 700 will take a long time to fulfill! -- starvation
 - **Distinction between Reads and Writes**
 - ▶ **Writes** are asynchronous: kernel copies data in buffer and process proceed
 - ▶ **Reads**: process must wait until requested data are delivered before proceeding
 - ▶ A bunch of large writes can starve out a read!

Deadline scheduler

- Uses three queues
 - Incoming requests go to **sorted elevator queue**
 - **Read** requests also go to tail of a **FIFO queue**
 - **Write** requests also go to tail of a **FIFO queue**
- Each request has an expiration time
 - (0.5 s for read, 5 s for write)
- Ordinarily dispatches from the sorted queue.
- When a request is satisfied, it is removed from the head of the sorted queue and also from the appropriate FIFO queue.
- However, when the item at the head of one of the FIFO queues bypass its expiration time, then the scheduler next dispatches from that FIFO queue, taking the expired request, plus the next few requests from the queue.
- As each request is dispatched, it is also removed from the sorted queue

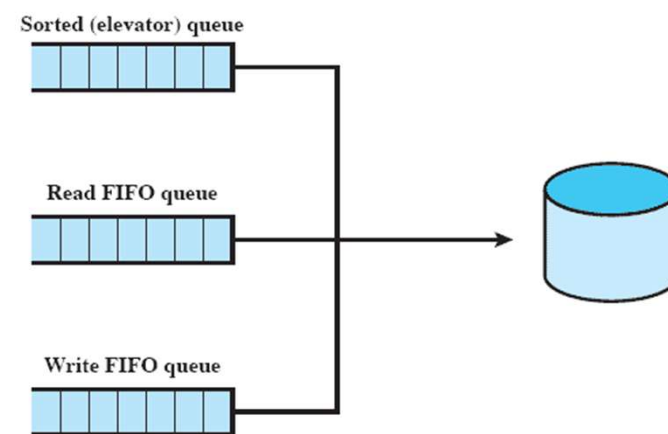


Figure 11.14 The Linux Deadline I/O Scheduler

Overcomes

- starvation problem
- Read vs Write problem

Anticipatory I/O scheduler

- Elevator and deadline scheduling can be counterproductive if there are numerous synchronous read requests:
 - Because of principle of locality, it is likely that successive reads from the same process will be to disk blocks that are near one another.
 - If the scheduler were to delay a short period of time after satisfying a read request, to see if a new nearby read request is made, the overall performance of the system could be enhanced.
- Anticipatory scheduler is superimposed on deadline scheduler
 - Delay a short period of time after satisfying a read request to see if a new nearby request can be made (6 ms)
 - During this delay, there is a chance that the process that issued the last read will issue another read request to the same region of the disk.
 - ▶ If so, that request will be serviced immediately.
 - ▶ If no such read request occurs, the scheduler resumes using the deadline scheduling algorithm.

Windows I/O Manager

- ❑ The I/O manager is responsible for all I/O for the operating system
- ❑ It provides a uniform interface that all types of drivers can call.
- ❑ The I/O manager works closely with four kernel components

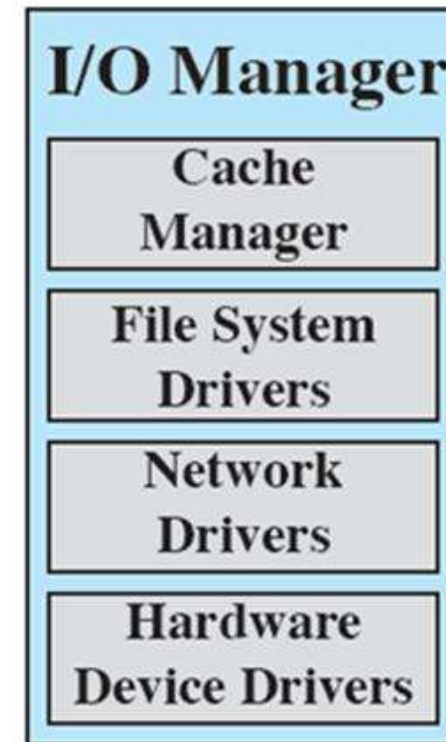


Figure 11.15 Windows I/O Manager

Windows I/O

- The I/O manager works closely with 4 components:

- **Cache manager**

- ▶ Cache manager handles file caching for all file systems
- ▶ Cache manager records updates only in cache, not on disks
- ▶ I/O manager: Lazy writer thread – periodically batches updates to write to disk – for IO efficiency

- **File system drivers**

- ▶ IO Manager treats a File System driver as an ordinary device driver
 - It routes I/O requests for file system volumes to the appropriate software driver for that volume.

- **Network drivers** - implemented as software drivers rather than part of the Windows Executive

- **Hardware device drivers**

- ▶ These software drivers access the hardware registers of the peripheral devices using entry points in the kernel's Hardware Abstraction Layer.
- ▶ A set of these routines exists for every platform that Windows supports

Asynchronous and Synchronous I/O

- Windows offers two modes of I/O operation
- **Synchronous:** Application blocks until I/O finishes
- **Asynchronous:**
 - Application continues and is informed of completion later
 - Asynchronous mode is used whenever possible to optimize performance
 - Application needs a signal when the operation is complete
- Windows offer several **signalling techniques**
 - Signal the file object
 - ▶ Simple to handle
 - ▶ Cannot distinguish multiple requests on different portions of the same file
 - Signal a event kernel object
 - ▶ The thread creates an event for each request.
 - ▶ The thread can wait on one of these requests or on an entire collection of requests, allowing simultaneous I/O requests against a single device or file
 - Asynchronous Procedure Call: I/O manager places results in APC queue
 - I/O completion ports
 - Use a pool of threads to handle requests

RAID Configurations

- ❑ Windows supports two RAID configurations:
- ❑ **Hardware RAID**
 - ❑ Separate physical disks combined into one or more logical disks by the disk controller
- ❑ **Software RAID**
 - ❑ Noncontiguous disk space combined into one or more logical partitions
 - ❑ implements RAID functionality as part of the operating system and can be used with any set of multiple disks.
 - ❑ RAID 0, 1 and RAID 5 are supported.
 - ❑ In the case of RAID 1 (disk mirroring), the two disks containing the primary and mirrored partitions may be on the same disk controller or different disk controllers.